

Log File Anomaly Explainer – AI Notes

Project Title

Log File Anomaly Explainer

1. Project Objective

The objective of this project is to automate log file analysis by identifying suspicious events, errors, warnings, security threats, and abnormal system behavior.

The system reduces the effort required for manual log monitoring and helps users quickly understand issues through automated explanations and severity classification.

2. Problem Statement

Modern systems generate thousands of log entries every day.

Manual log analysis has several challenges:

- Time-consuming process
- Human errors during analysis
- Difficult to identify critical issues quickly
- Security threats may go unnoticed

This project solves the problem by automatically detecting anomalies and providing understandable explanations.

3. Project Description

Log File Anomaly Explainer is a Flask-based web application developed using Python.

The user uploads a log file through a web interface.

The application reads the log entries, detects anomalies using predefined patterns, classifies the severity level, generates explanations, and displays the results on a dashboard.

4. Technologies Used

Programming Language

- Python

Backend Framework

- Flask

Frontend

- HTML
- CSS

Development Tools

- VS Code
- Git
- GitHub

Libraries

- Flask
 - Pandas
 - Pytest
 - Google Generative AI (Prepared for future integration)
-

5. System Workflow

User Uploads Log File



File Processing Module



Anomaly Detection Module



Severity Classification Module



AI Explanation Module



Result Dashboard

6. Working Principle

Step 1: Upload Log File

The user uploads a .txt or .log file through the web interface.

Step 2: Read Log Data

The system reads the uploaded file line by line.

Step 3: Detect Anomalies

The anomaly detector searches for predefined keywords such as:

- ERROR
- WARNING
- FAILED
- FAIL
- CRITICAL
- SECURITY
- UNAUTHORIZED
- ACCESS DENIED
- TIMEOUT

Step 4: Severity Assignment

Detected anomalies are categorized into severity levels.

Step 5: Explanation Generation

The AI explanation module generates meaningful descriptions for detected anomalies.

Step 6: Display Results

The results are shown on a web dashboard.

7. Severity Levels

High Severity

Used for critical security or system threats.

Examples:

- Unauthorized Access
 - Security Breach
 - Critical Failures
-

Medium Severity

Used for operational failures.

Examples:

- Database Errors
 - Failed Login Attempts
 - Application Errors
-

Low Severity

Used for warnings and monitoring events.

Examples:

- Timeout Events
 - Warning Messages
 - Resource Usage Alerts
-

8. Key Features

- Log file upload support
- Automatic anomaly detection
- Severity classification
- AI-generated explanations
- User-friendly dashboard
- Fast processing
- Simple architecture
- Easy deployment

9. Advantages

- Reduces manual effort
- Faster issue identification
- Improves system monitoring
- Helps detect security threats
- Easy to use
- Lightweight application

10. Limitations

- Uses rule-based detection
- No real-time monitoring
- No machine learning model
- Supports only text-based log files

11. Future Enhancements

The project can be enhanced by adding:

- Google Gemini API Integration
 - Machine Learning Based Detection
 - Real-Time Log Monitoring
 - Email Notifications
 - PDF Report Generation
 - Cloud Deployment
 - Advanced Security Analytics
 - Database Integration
-